The Substitution Model

Concepts of Programming Languages

CAS CS 320

```
<prog> ::= { <stmt> ; }
<stmt> ::= rot90 | refX | refY
```

Practice Problem

$$\frac{}{(s,\ \texttt{rot90;}\ P) \longrightarrow (s \text{ rotated 90 deg. clockwise},\ P)}$$

$$\frac{}{(s,\ \texttt{refX;}\ P) \longrightarrow (s \text{ reflected across x-axis},\ P)}$$

$$\frac{}{(s,\ \texttt{refY;}\ P) \longrightarrow (s \text{ reflected across y-axis},\ P)}$$

What does (△, `rot90; refY; rot90; refX;`) *evaluate to? Give a sequence of single step reductions (you do not need to give the full multi-step derivation)*

Answer

```
<prog> ::= { <stmt> ; }
<stmt> ::= rot90 | refX | refY
```

(△, rot90; refY; rot90; refX;)

(△, r90; rY; r90; rX;) →

(▷, rY; r90; rX;) →

(◁, r90; rX;) →

(△, rX;) →

(▽, ε)

empty program

Outline

- Look formally at the **lambda calculus** and its semantics
- Discuss **substitution** and the pitfalls to avoid
- Demo an **implementation** of the lambda calculus

Recap

Recall: High-Level

$$(\underbrace{S}_{\text{state}}, \overbrace{p}^{\text{program}}) \xrightarrow[\text{transformed state}]{} (S', \overbrace{p'}^{\text{transformed program}})$$

Recall: High-Level

$$(\underbrace{S}_{\text{state}}, \overbrace{p}^{\text{program}}) \xrightarrow[\text{transformed state}]{} (S', \overbrace{p'}^{\text{transformed program}})$$

When we define the small-step semantics of PL, we need to define three things:

Recall: High-Level

$$(\underbrace{S}_{\text{state}}, \overbrace{p}^{\text{program}}) \xrightarrow[\text{transformed state}]{} (S', \overbrace{p'}^{\text{transformed program}})$$

When we define the small-step semantics of PL, we need to define three things:

» What kind of **state** are we manipulating?

Recall: High-Level

$$(\underbrace{S}_{\text{state}}, \overbrace{p}^{\text{program}}) \xrightarrow{} (\underbrace{S'}_{\text{transformed state}}, \overbrace{p'}^{\text{transformed program}})$$

When we define the small-step semantics of PL, we need to define three things:

» What kind of **state** are we manipulating?

» What **rules** describe how to transform configurations?

Recall: High-Level

$$(\underbrace{S}_{\text{state}}, \overbrace{p}^{\text{program}}) \xrightarrow{} (\underbrace{S'}_{\text{transformed state}}, \overbrace{p'}^{\text{transformed program}})$$

When we define the small-step semantics of PL, we need to define three things:

» What kind of **state** are we manipulating?

» What **rules** describe how to transform configurations?

» What are the **values** of our PL (i.e., when are we done reducing)?

Recall: Example

```
<expr> ::= ( <op> <expr> <expr> )
         | <bool> | <int>
<op>   ::= add | sub | eq
<bool> ::= true | false
<int>  ::= ...
```

Recall: Example

```
<expr> ::= ( <op> <expr> <expr> )
         | <bool> | <int>
<op>   ::= add | sub | eq
<bool> ::= true | false
<int>  ::= ...
```

State: ∅

Recall: Example

```
<expr> ::= ( <op> <expr> <expr> )
         | <bool> | <int>
<op>   ::= add | sub | eq
<bool> ::= true | false
<int>  ::= ...
```

State: $\varnothing$

Rules:

$$\frac{e_1 \longrightarrow e_1'}{(\texttt{add}\ e_1\ e_2) \longrightarrow (\texttt{add}\ e_1'\ e_2)}\ \texttt{add-left}$$

$$\frac{n\ \texttt{is a number} \qquad e_2 \longrightarrow e_2'}{(\texttt{add}\ n\ e_2) \longrightarrow (\texttt{add}\ n\ e_2')}\ \texttt{add-right}$$

$$\frac{n_1\ \texttt{is a number} \qquad n_2\ \texttt{is a number}}{(\texttt{add}\ n_1\ n_2) \longrightarrow n_1 + n_2}\ \texttt{add-ok}$$

$$\frac{e_1 \longrightarrow e_1'}{(\texttt{sub}\ e_1\ e_2) \longrightarrow (\texttt{sub}\ e_1'\ e_2)}\ \texttt{sub-left}$$

$$\frac{n\ \texttt{is a number} \qquad e_2 \longrightarrow e_2'}{(\texttt{sub}\ n\ e_2) \longrightarrow (\texttt{sub}\ n\ e_2')}\ \texttt{sub-right}$$

$$\frac{n_1\ \texttt{is a number} \qquad n_2\ \texttt{is a number}}{(\texttt{sub}\ n_1\ n_2) \longrightarrow n_1 - n_2}\ \texttt{sub-ok}$$

Recall: Example

```
<expr> ::= ( <op> <expr> <expr> )
         | <bool> | <int>
<op>   ::= add | sub | eq
<bool> ::= true | false
<int>  ::= ...
```

State: $\varnothing$

Rules:

$$\frac{e_1 \longrightarrow e_1'}{(\texttt{add}\ e_1\ e_2) \longrightarrow (\texttt{add}\ e_1'\ e_2)}\ \texttt{add-left}$$

$$\frac{n\ \texttt{is a number} \qquad e_2 \longrightarrow e_2'}{(\texttt{add}\ n\ e_2) \longrightarrow (\texttt{add}\ n\ e_2')}\ \texttt{add-right}$$

$$\frac{n_1\ \texttt{is a number} \qquad n_2\ \texttt{is a number}}{(\texttt{add}\ n_1\ n_2) \longrightarrow n_1 + n_2}\ \texttt{add-ok}$$

$$\frac{e_1 \longrightarrow e_1'}{(\texttt{sub}\ e_1\ e_2) \longrightarrow (\texttt{sub}\ e_1'\ e_2)}\ \texttt{sub-left}$$

$$\frac{n\ \texttt{is a number} \qquad e_2 \longrightarrow e_2'}{(\texttt{sub}\ n\ e_2) \longrightarrow (\texttt{sub}\ n\ e_2')}\ \texttt{sub-right}$$

$$\frac{n_1\ \texttt{is a number} \qquad n_2\ \texttt{is a number}}{(\texttt{sub}\ n_1\ n_2) \longrightarrow n_1 - n_2}\ \texttt{sub-ok}$$

Values: `<int>` (i.e., numbers)

The Lambda Calculus

High-Level View

```
(fun x -> x x)(fun x -> x x)
```

lambda term called Ω

High-Level View

```
(fun x -> x x)(fun x -> x x)
```

lambda term called Ω

The **lambda calculus** is the *simplest functional programming language*. It has:

High-Level View

```
(fun x -> x x)(fun x -> x x)
```

lambda term called Ω

The **lambda calculus** is the *simplest functional programming language*. It has:

» variables

High-Level View

```
(fun x -> x x)(fun x -> x x)
```

lambda term called Ω

The **lambda calculus** is the *simplest functional programming language*. It has:

» variables
» anonymous functions

High-Level View

```
(fun x -> x x)(fun x -> x x)
```

lambda term called Ω

The **lambda calculus** is the *simplest functional programming language*. It has:

- variables
- anonymous functions
- function application

High-Level View

```
(fun x -> x x)(fun x -> x x)
```

lambda term called Ω

The **lambda calculus** is the *simplest functional programming language*. It has:

» variables
» anonymous functions
» function application

It's **untyped**, so *anything* can be applied to *anything*

demo

(OCaml and Python)

History

History

The lambda calculus was introduced by **Alonzo Church** in the 1930s

History

The lambda calculus was introduced by **Alonzo Church** in the 1930s

Church was trying to give a *foundation of mathematics* (did not succeed) and extracted from that work the lambda calculus

History

The lambda calculus was introduced by **Alonzo Church** in the 1930s

Church was trying to give a *foundation of mathematics* (did not succeed) and extracted from that work the lambda calculus

The lambda calculus **as powerful** as every model of computation (Turing Machines, Register Machines, etc.)

Syntax

```
<expr> ::= fun <var> -> <expr>
         | <var>
         | <expr> <expr>
         | ( <expr> )
<var>  ::= a | b | ... | y | z
```

Syntax

```
<expr> ::= fun <var> -> <expr>
         | <var>
         | <expr> <expr>
         | ( <expr> )
<var>  ::= a | b | ... | y | z
```

This presentation is technically ambiguous (why?)

Syntax

```
<expr> ::= fun <var> -> <expr>
         | <var>
         | <expr> <expr>
         | ( <expr> )
<var>  ::= a | b | ... | y | z
```

This presentation is technically ambiguous (why?)

We assume application is **left-associative** and has higher precedence than the anonymous function syntax

Syntax (Unambiguous)

```
<expr>  ::= fun <var> -> <expr>
          | <expr2> { <expr2> }
<expr2> ::= <var> | ( <expr> )
<var>   ::= a | b | ... | y | z
```

In this grammar we can only use variables or functions in parentheses in applications

Syntax (Mathematical)

```
<expr> ::= λ<var>.<expr>
         | <var>
         | <expr><expr>
```

In mathematical settings, we use more compact syntax

Parentheses, precedence, and variables are often left implicit

Warning: Get used to λ

$$\lambda x . e \quad \equiv \quad \texttt{fun}\ x\ \texttt{->}\ e$$

We will use these syntaxes interchangeably starting now

These are the same thing, get used to it

Values

<val> ::= λ<var>.<expr>

Values

$$\langle val \rangle ::= \lambda \langle var \rangle . \langle expr \rangle$$

In arithmetic, values are **numbers**

Values

$$\langle val \rangle ::= \lambda \langle var \rangle . \langle expr \rangle$$

In arithmetic, values are **numbers**

In the lambda calculus, values are **functions**

Values

$$\langle val \rangle ::= \lambda \langle var \rangle . \langle expr \rangle$$

In arithmetic, values are **numbers**

In the lambda calculus, values are **functions**

(We often use BNF syntax to specify our values)

Small-Step Semantics

$$(1)\ \frac{e_1 \longrightarrow e_1'}{e_1 e_2 \longrightarrow e_1' e_2}$$

$$(2)\ \frac{e_2 \longrightarrow e_2'}{(\lambda x\,.\,e_1)e_2 \longrightarrow (\lambda x\,.\,e_1)e_2'}$$

$$(3)\ \frac{}{(\lambda x\,.\,e)(\lambda y\,.\,e') \longrightarrow [(\lambda y\,.\,e')/x]e}$$

Small-Step Semantics

$$(1)\ \frac{e_1 \longrightarrow e_1'}{e_1 e_2 \longrightarrow e_1' e_2}$$

$$(2)\ \frac{e_2 \longrightarrow e_2'}{(\lambda x\,.\,e_1)e_2 \longrightarrow (\lambda x\,.\,e_1)e_2'}$$

$$(3)\ \frac{}{(\lambda x\,.\,e)(\lambda y\,.\,e') \longrightarrow [(\lambda y\,.\,e')/x]e}$$

1. We can reduce the LHS of an application

Small-Step Semantics

$$(1)\ \frac{e_1 \longrightarrow e_1'}{e_1 e_2 \longrightarrow e_1' e_2}$$

$$(2)\ \frac{e_2 \longrightarrow e_2'}{\boxed{(\lambda x\,.\,e_1)} e_2 \longrightarrow (\lambda x\,.\,e_1) e_2'}$$

value

$$(3)\ \frac{}{(\lambda x\,.\,e)(\lambda y\,.\,e') \longrightarrow [(\lambda y\,.\,e')/x]e}$$

1. We can reduce the LHS of an application

2. We can reduce the RHS of an application *if the LHS is already a function*

Small-Step Semantics

$$(1)\ \frac{e_1 \longrightarrow e_1'}{e_1 e_2 \longrightarrow e_1' e_2}$$

$$(2)\ \frac{e_2 \longrightarrow e_2'}{(\lambda x\,.\,e_1)e_2 \longrightarrow (\lambda x\,.\,e_1)e_2'}$$

$$(3)\ \frac{}{(\lambda x\,.\,e)(\lambda y\,.\,e') \longrightarrow [(\lambda y\,.\,e')/x]e}$$

1. We can reduce the LHS of an application

2. We can reduce the RHS of an application *if the LHS is already a function*

3. We can apply a function to another function by **substitution.** This is also called **β-reduction**

Example

$$(\lambda \boxed{f} . \lambda x . \boxed{f} x)\boxed{(\lambda y . y)} \xrightarrow{(3)}$$

$$\underbrace{\lambda x . \boxed{(\lambda y . y)\, x}}_{\text{value}}$$

(1) $$\frac{e_1 \longrightarrow e_1'}{e_1 e_2 \longrightarrow e_1' e_2}$$

(2) $$\frac{e_2 \longrightarrow e_2'}{(\lambda x . e_1) e_2 \longrightarrow (\lambda x . e_1) e_2'}$$

(3) $$\frac{}{(\lambda x . e)(\lambda y . e') \longrightarrow [(\lambda y . e')/x]e}$$

Small-Step Semantics (Another Form)

$$(1)\ \frac{e_1 \longrightarrow e_1'}{e_1 e_2 \longrightarrow e_1' e_2}$$

$$(2)\ \frac{}{(\lambda x\,.\,e)e' \longrightarrow [e'/x]e}$$

Small-Step Semantics (Another Form)

$$(1)\ \frac{e_1 \longrightarrow e_1'}{e_1 e_2 \longrightarrow e_1' e_2}$$

$$(2)\ \frac{}{(\lambda x\,.\,e)e' \longrightarrow [e'/x]e}$$

1. We can reduce the LHS of an application

Small-Step Semantics (Another Form)

$$(1)\ \frac{e_1 \longrightarrow e_1'}{e_1 e_2 \longrightarrow e_1' e_2}$$

$$(2)\ \frac{}{(\lambda x\,.\,e)e' \longrightarrow [e'/x]e}$$

1. We can reduce the LHS of an application
2. We can apply a function to *any* expression via substitution

Example

$$(1)\ \frac{e_1 \longrightarrow e_1'}{e_1 e_2 \longrightarrow e_1' e_2} \qquad (2)\ \frac{}{(\lambda x\,.\,e)e' \longrightarrow \underbrace{[e'/x]e}_{\text{subs. } e' \text{ for } x \text{ in } e}}$$

$$(\lambda \boxed{x}\,.\,y)\boxed{((\lambda z\,.\,z)(\lambda w\,.\,w))} \xrightarrow{(2)} \underbrace{y}_{\text{not a value}}$$

stuck term

Recall: Values

Recall: Values

When evaluating, there are **three** "end" cases:

Recall: Values

When evaluating, there are **three** "end" cases:

» **value:** we reach the end of our computation and the value of our program

`(fun x -> x) (2 + 3)` $\rightarrow^{\star}$ `5`

Recall: Values

When evaluating, there are **three** "end" cases:

» **value:** we reach the end of our computation and the value of our program

» **stuck:** we reach an expression that cannot be reduced, but that is not a value

`(fun x -> x) (2 + 3)` $\rightarrow^{\star}$ `5`

`y (fun x -> x)` $\not\rightarrow$

Recall: Values

When evaluating, there are **three** "end" cases:

» **value:** we reach the end of our computation and the value of our program

`(fun x -> x) (2 + 3)` $\rightarrow^{\star}$ `5`

» **stuck:** we reach an expression that cannot be reduced, but that is not a value

`y (fun x -> x)` $\not\rightarrow$

» **diverge:** the computation never reaches a point where the expression is not reducible

`(fun x -> x x) (fun x -> x x)` $\rightarrow^{\star}$
`(fun x -> x x) (fun x -> x x)`

Stuck Terms

<val> ::= λ<var>.<expr>

$$\frac{e_1 \longrightarrow e_1'}{e_1 e_2 \longrightarrow e_1' e_2} \qquad \frac{}{(\lambda x\,.\,e)e' \longrightarrow [e'/x]e}$$

$$(\lambda \boxed{x}\,.\,yx)\boxed{(\lambda x\,.\,x)} \xrightarrow{(2)} y\,(\lambda x\,.\,x)$$

Based on our operational semantics, it's possible for the above expression to reduce to a value

Non-Termination

$$\frac{e_1 \longrightarrow e_1'}{e_1 e_2 \longrightarrow e_1' e_2} \qquad \frac{}{(\lambda x\,.\,e)e' \longrightarrow [e'/x]e}$$

$$(\lambda \boxed{x}\,.\,xx)\boxed{(\lambda x\,.\,xx)} \xrightarrow{(2)}$$

$$\Omega$$

$$(\lambda x.\,xx)(\lambda x.\,xx) \xrightarrow{(2)}$$

$$(\lambda x.\,xx)(\lambda x.\,xx) \xrightarrow{(2)} \cdots$$

And unlike with arithmetic, it's now possible to define expressions which **do not terminate.** These expression do not have values, but also don't get stuck

Big-Step Semantics

$$(1)\ \frac{}{\lambda x\,.\,e \Downarrow \lambda x\,.\,e}$$

$$(2)\ \frac{e_1 \Downarrow \lambda x\,.\,e \qquad e_2 \Downarrow v_2 \qquad [v_2/x]e \Downarrow v}{e_1 e_2 \Downarrow v}$$

Big-Step Semantics

$$(1)\ \frac{}{\lambda x\,.\,e \Downarrow \lambda x\,.\,e}$$

$$(2)\ \frac{e_1 \Downarrow \lambda x\,.\,e \qquad e_2 \Downarrow v_2 \qquad [v_2/x]e \Downarrow v}{e_1 e_2 \Downarrow v}$$

1. A function evaluates to a function value (itself)

Big-Step Semantics

$$(1)\ \frac{}{\lambda x\,.\,e \Downarrow \lambda x\,.\,e}$$

$$(2)\ \frac{e_1 \Downarrow \lambda x\,.\,e \qquad e_2 \Downarrow v_2 \qquad [v_2/x]e \Downarrow v}{e_1 e_2 \Downarrow v}$$

1. A function evaluates to a function value (itself)

2. If e_1 evaluates to the function $\lambda x\,.\,e$ and e_2 evaluates to the value v_2 and e with v_2 substituted for x evaluates to v, then the application $e_1 e_2$ evaluates to v

Big-Step Semantics (Another Form)

$$\frac{}{\lambda x\,.\,e \Downarrow \lambda x\,.\,e}$$

$$\frac{e_1 \Downarrow \lambda x\,.\,e \qquad [e_2/x]e \Downarrow v}{e_1 e_2 \Downarrow v}$$

These are the same rules as before except we're not required to evaluate e_2 first

Practice Problem

$$(\lambda x\,.\,\lambda y\,.\,y)((\lambda z\,.\,z)(\lambda q\,.\,q)) \Downarrow \lambda y\,.\,y$$

Give a derivation of the above judgment in both versions of the big-step semantics

$$\frac{\dfrac{}{\lambda x\,.\,e \Downarrow \lambda x\,.\,e} \qquad \dfrac{}{\lambda x\,.\,e \Downarrow \lambda x\,.\,e} \qquad \dfrac{}{[v_2/x]e \Downarrow v}}{e_1 e_2 \Downarrow v}$$

$$\frac{\dfrac{}{\lambda x\,.\,e \Downarrow \lambda x\,.\,e} \qquad \dfrac{}{[e_2/x]e \Downarrow v}}{e_1 e_2 \Downarrow v}$$

Answer

$$(\lambda x\,.\,\lambda y\,.\,y)((\lambda z\,.\,z)(\lambda q\,.\,q)) \Downarrow \lambda y\,.\,y$$

(right system)

$$\dfrac{\dfrac{}{\lambda x \lambda y.y \Downarrow \lambda x.\boxed{\lambda y.y}} \qquad \dfrac{}{\lambda y.y \Downarrow \lambda y.y}}{(\lambda x.\lambda y.y)\boxed{((\lambda z.z)(\lambda q.q))} \Downarrow \lambda y.y}$$

Call-by-Value vs. Call-by-Name

$$\frac{e_1 \Downarrow \lambda x\,.\,e_1' \qquad e_2 \Downarrow v_2 \qquad [v_2/x]e_1' \Downarrow v}{e_1 e_2 \Downarrow v}\ \texttt{(CBV)}$$

$$\frac{e_1 \Downarrow \lambda x\,.\,e_1' \qquad [e_2/x]e_1' \Downarrow v}{e_1 e_2 \Downarrow v}\ \texttt{(CBN)}$$

Call-by-Value vs. Call-by-Name

$$\frac{e_1 \Downarrow \lambda x\,.\,e_1' \qquad e_2 \Downarrow v_2 \qquad [v_2/x]e_1' \Downarrow v}{e_1 e_2 \Downarrow v}\ \text{(CBV)}$$

$$\frac{e_1 \Downarrow \lambda x\,.\,e_1' \qquad [e_2/x]e_1' \Downarrow v}{e_1 e_2 \Downarrow v}\ \text{(CBN)}$$

The two versions of semantics we've given correspond to **call-by-value** (CBV) and **call-by-name** (CBN)**.** These are **evaluation strategies** for functional languages

Call-by-Value vs. Call-by-Name

$$\frac{e_1 \Downarrow \lambda x\,.\,e_1' \qquad e_2 \Downarrow v_2 \qquad [v_2/x]e_1' \Downarrow v}{e_1 e_2 \Downarrow v}\ \text{(CBV)}$$

$$\frac{e_1 \Downarrow \lambda x\,.\,e_1' \qquad [e_2/x]e_1' \Downarrow v}{e_1 e_2 \Downarrow v}\ \text{(CBN)}$$

The two versions of semantics we've given correspond to **call-by-value** (CBV) and **call-by-name** (CBN)**.** These are **evaluation strategies** for functional languages

<u>CBV:</u> evaluate the argument of a function *before* substituting it in the function

Call-by-Value vs. Call-by-Name

$$\frac{e_1 \Downarrow \lambda x\,.\,e_1' \qquad e_2 \Downarrow v_2 \qquad [v_2/x]e_1' \Downarrow v}{e_1 e_2 \Downarrow v}\ \text{(CBV)}$$

$$\frac{e_1 \Downarrow \lambda x\,.\,e_1' \qquad [e_2/x]e_1' \Downarrow v}{e_1 e_2 \Downarrow v}\ \text{(CBN)}$$

The two versions of semantics we've given correspond to **call-by-value** (CBV) and **call-by-name** (CBN)**.** These are **evaluation strategies** for functional languages

<u>CBV:</u> evaluate the argument of a function *before* substituting it in the function

<u>CBN:</u> substitute the expression *directly* into the function

Benefits of CBV

$$\frac{}{\lambda x\,.\,e \Downarrow \lambda x\,.\,e} \qquad \frac{e_1 \Downarrow \lambda x\,.\,e_1' \qquad e_2 \Downarrow v_2 \qquad [v_2/x]e_1' \Downarrow v}{e_1 e_2 \Downarrow v}$$

$$(\lambda x\,.\,x + x + x + x)e$$

Benefits of CBV

$$\frac{}{\lambda x\,.\,e \Downarrow \lambda x\,.\,e} \qquad \frac{e_1 \Downarrow \lambda x\,.\,e_1' \qquad e_2 \Downarrow v_2 \qquad [v_2/x]e_1' \Downarrow v}{e_1 e_2 \Downarrow v}$$

$$(\lambda x\,.\,x + x + x + x)e$$

If we compute the value of an argument before substituting it into the expression, we only have to compute the expression *once*

Benefits of CBV

$$\frac{}{\lambda x\,.\,e \Downarrow \lambda x\,.\,e} \qquad \frac{e_1 \Downarrow \lambda x\,.\,e_1' \qquad e_2 \Downarrow v_2 \qquad [v_2/x]e_1' \Downarrow v}{e_1 e_2 \Downarrow v}$$

$$(\lambda x\,.\,x + x + x + x)e$$

If we compute the value of an argument before substituting it into the expression, we only have to compute the expression *once*

This is good if the variable appears several times in the body of our function

$$\frac{}{\lambda x\,.\,e \Downarrow \lambda x\,.\,e} \qquad \frac{e_1 \Downarrow \lambda x\,.\,e_1' \qquad e_2 \Downarrow v_2 \qquad [v_2/x]e_1' \Downarrow v}{e_1 e_2 \Downarrow v}$$

Benefits of CBV

$$(\lambda x\,.\,x + x + x + x)e$$

If we compute the value of an argument before substituting it into the expression, we only have to compute the expression *once*

This is good if the variable appears several times in the body of our function

This is also called **eager**, or **applicative**, or **strict** evaluation (and is what OCaml does)

Benefits of CBN

$$\frac{}{\lambda x\,.\,e \Downarrow \lambda x\,.\,e}$$

$$\frac{e_1 \Downarrow \lambda x\,.\,e_1' \qquad [e_2/x]e_1' \Downarrow v}{e_1 e_2 \Downarrow v}$$

$$(\lambda x\,.\,\lambda y\,.\,x)e_1 e_2$$

Benefits of CBN

$$\frac{}{\lambda x\,.\,e \Downarrow \lambda x\,.\,e}$$

$$\frac{e_1 \Downarrow \lambda x\,.\,e_1' \qquad [e_2/x]e_1' \Downarrow v}{e_1 e_2 \Downarrow v}$$

$$(\lambda x\,.\,\lambda y\,.\,x)e_1 e_2$$

If a variables doesn't appear in our function, then the argument is *not evaluated at all*

Benefits of CBN

$$\frac{}{\lambda x\,.\,e \Downarrow \lambda x\,.\,e}$$

$$\frac{e_1 \Downarrow \lambda x\,.\,e_1' \qquad [e_2/x]e_1' \Downarrow v}{e_1 e_2 \Downarrow v}$$

$$(\lambda x\,.\,\lambda y\,.\,x)e_1 e_2$$

If a variables doesn't appear in our function, then the argument is *not evaluated at all*

If an argument is only seldomly used, it will only be computed when it is used (e.g, if its computed in a branch of an if-expression that is almost never reached)

Benefits of CBN

$$\frac{}{\lambda x\,.\,e \Downarrow \lambda x\,.\,e}$$

$$\frac{e_1 \Downarrow \lambda x\,.\,e_1' \qquad [e_2/x]e_1' \Downarrow v}{e_1 e_2 \Downarrow v}$$

$$(\lambda x\,.\,\lambda y\,.\,x)e_1 e_2$$

If a variables doesn't appear in our function, then the argument is *not evaluated at all*

If an argument is only seldomly used, it will only be computed when it is used (e.g, if its computed in a branch of an if-expression that is almost never reached)

Aside. It's possible to simulate CBN in CBV. Think about it for a bit, ask me after if you're interested

Side Effects

```
let f x = x + x in
let y =
  let _ = print_int 2 in
  2
in f y
```

Side Effects

```
let f x = x + x in
let y =
  let _ = print_int 2 in
  2
in f y
```

What does this program print? ***It depends on if we're using CBN or CBV evaluation***

Side Effects

```
let f x = x + x in
let y =
  let _ = print_int 2 in
  2
in f y
```

What does this program print? ***It depends on if we're using CBN or CBV evaluation***

Definition. *(informal)* A **side effect** refers to something that happens during the evaluation of a program that is not a part of the formal semantics, e.g., printing to the console

Side Effects

```
let f x = x + x in
let y =
  let _ = print_int 2 in
  2
in f y
```

What does this program print? ***It depends on if we're using CBN or CBV evaluation***

Definition. *(informal)* A **side effect** refers to something that happens during the evaluation of a program that is not a part of the formal semantics, e.g., printing to the console

Different evaluation strategies yield different side-effectful behavior!

Aside: Evaluation Strategies

Aside: Evaluation Strategies

There are *a lot* more evaluation strategies, all of which optimize *something*

Aside: Evaluation Strategies

There are *a lot* more evaluation strategies, all of which optimize *something*

Haskell uses **lazy** evaluation also called **call-by-need**

Aside: Evaluation Strategies

There are *a lot* more evaluation strategies, all of which optimize *something*

Haskell uses **lazy** evaluation also called **call-by-need**

In languages with pointers we also often have the option to use **call-by-reference** evaluation or **call-by-sharing**

Aside: Evaluation Strategies

There are *a lot* more evaluation strategies, all of which optimize *something*

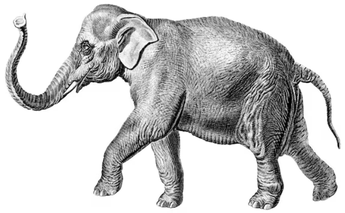
Haskell uses **lazy** evaluation also called **call-by-need**

In languages with pointers we also often have the option to use **call-by-reference** evaluation or **call-by-sharing**

*We will exclusively implement **call-by-value** (because, again, this is what OCaml does)*

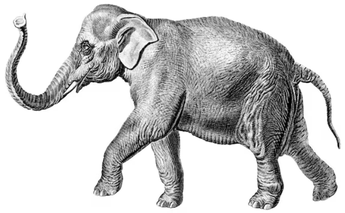
Substitution

Substitution



$$\frac{e_1 \Downarrow \lambda x\,.\,e \qquad e_2 \Downarrow v_2 \qquad [v_2/x]e \Downarrow v}{e_1 e_2 \Downarrow v}$$

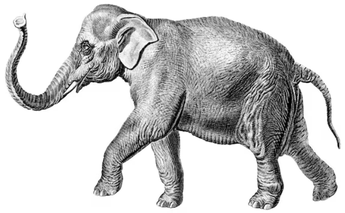
Substitution



$$\frac{e_1 \Downarrow \lambda x\,.\,e \qquad e_2 \Downarrow v_2 \qquad [v_2/x]e \Downarrow v}{e_1 e_2 \Downarrow v}$$

It's time to get more formal about **substitution**

Substitution

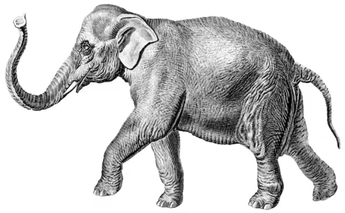


$$\frac{e_1 \Downarrow \lambda x\,.\,e \qquad e_2 \Downarrow v_2 \qquad [v_2/x]e \Downarrow v}{e_1 e_2 \Downarrow v}$$

It's time to get more formal about **substitution**

We've been able to get by on our intuitions for a while, but our intuitions won't help us *implement* substitution (which is *difficult*)

Substitution



$$\frac{e_1 \Downarrow \lambda x\,.\,e \qquad e_2 \Downarrow v_2 \qquad [v_2/x]e \Downarrow v}{e_1 e_2 \Downarrow v}$$

It's time to get more formal about **substitution**

We've been able to get by on our intuitions for a while, but our intuitions won't help us *implement* substitution (which is *difficult*)

We need to understand why...

Notation

$$[y/x](\lambda x \,.\, y)$$

Notation

$$[y/x](\lambda x\,.\,y)$$

We write $[v/x]e$ to mean e with v substituted in for x

Notation

$$[y/x](\lambda x\,.\,y)$$

We write $[v/x]e$ to mean e with v substituted in for x

Informally. *Replace every instance of* x *with* v

Notation

$$[y/x](\lambda x.\, y) = \lambda y.\, y$$

We write $[v/x]e$ to mean e with v substituted in for x

Informally. *Replace every instance of x with v*

Already things start to break down with this informal definition, e.g., consider the above substitution...

The Idea

$$[y/x](\lambda x \,.\, y)$$

The Idea

$$[y/x](\lambda x\,.\,y)$$

However we define substitution shouldn't *change the underlying behavior of a function*

The Idea

$$[y/x](\lambda x\,.\,y)$$

However we define substitution shouldn't *change the underlying behavior of a function*

The Key Point: A function does not depend on our choice of variable names

α-Equivalence

```
let x = 2 in x + 1
```

$=_\alpha$

```
let z = 2 in z + 1
```

OCaml

$$\lambda x\,.\,\lambda y\,.\,x =_\alpha \lambda v\,.\,\lambda w\,.\,v$$

λ-calculus

α-Equivalence

```
let x = 2 in x + 1
```

$=_\alpha$

```
let z = 2 in z + 1
```

OCaml

$$\lambda x\,.\,\lambda y\,.\,x =_\alpha \lambda v\,.\,\lambda w\,.\,v$$

λ-calculus

The **principle of name irrelevance** says that any two programs that are the same up to "renaming of variables" should behave exactly the same way (they are **α-equivalent**)

α-Equivalence

```
let x = 2 in x + 1
```

$=_\alpha$

```
let z = 2 in z + 1
```

OCaml

$$\lambda x . \lambda y . x =_\alpha \lambda v . \lambda w . v$$

λ-calculus

The **principle of name irrelevance** says that any two programs that are the same up to "renaming of variables" should behave exactly the same way (they are **α-equivalent**)

We say that a variable x is **bound** is an expression if it appears in the expression as $(\ldots \lambda x . e \ldots)$

α-Equivalence

```
let x = 2 in x + 1
```

$=_\alpha$

```
let z = 2 in z + 1
```

OCaml

$$\lambda x . \lambda y . x =_\alpha \lambda v . \lambda w . v$$

λ-calculus

The **principle of name irrelevance** says that any two programs that are the same up to "renaming of variables" should behave exactly the same way (they are **α-equivalent**)

We say that a variable x is **bound** is an expression if it appears in the expression as $(\ldots \lambda x . e \ldots)$

Substitution should preserve this

Definition (First Attempt)

$$[v/y]x = \begin{cases} v & x = y \\ x & \texttt{else} \end{cases} \tag{1}$$

$$[v/y](\lambda x\,.\,e) = \lambda x\,.\,[v/y]e \tag{2}$$

$$[v/y](e_1 e_2) = ([v/y]e_1)([v/y]e_2) \tag{3}$$

Definition (First Attempt)

$$[v/y]x = \begin{cases} v & x = y \\ x & \texttt{else} \end{cases} \tag{1}$$

$$[v/y](\lambda x\,.\,e) = \lambda x\,.\,[v/y]e \tag{2}$$

$$[v/y](e_1 e_2) = ([v/y]e_1)([v/y]e_2) \tag{3}$$

1. Replace every y with v, leave other variables

Definition (First Attempt)

$$[v/y]x = \begin{cases} v & x = y \\ x & \texttt{else} \end{cases} \tag{1}$$

$$[v/y](\lambda x\,.\,e) = \lambda x\,.\,[v/y]e \tag{2}$$

$$[v/y](e_1 e_2) = ([v/y]e_1)([v/y]e_2) \tag{3}$$

1. Replace every y with v, leave other variables
2. Replace y with v in the body of a function

Definition (First Attempt)

$$[v/y]x = \begin{cases} v & x = y \\ x & \texttt{else} \end{cases} \tag{1}$$

$$[v/y](\lambda x\,.\,e) = \lambda x\,.\,[v/y]e \tag{2}$$

$$[v/y](e_1 e_2) = ([v/y]e_1)([v/y]e_2) \tag{3}$$

1. Replace every y with v, leave other variables
2. Replace y with v in the body of a function
3. Replace y with v in both subexpressions of an application

(This is an example of an *inductive definition*)

$$[v/y]x = \begin{cases} v & x = y \\ x & \texttt{else} \end{cases}$$

$$[v/y](\lambda x\,.\,e) = \lambda x\,.\,[v/y]e$$

$$[v/y](e_1 e_2) = ([v/y]e_1)([v/y]e_2)$$

Problem Case I

α

$$[y/x](\lambda w.w) = \qquad [y/x](\lambda x\,.\,x) =$$

$$\lambda w.[y/x]w = \qquad \lambda x.[y/x]x =$$

$$\lambda w.w \neq \lambda x.y$$

α

We shouldn't be allowed to substitute x if it's the argument of a function

This may *change the behavior* of a function

Definition (Second Attempt)

$$[v/y]x = \begin{cases} v & x = y \\ x & \texttt{else} \end{cases}$$

$$[v/y](\lambda x\,.\,e) = \begin{cases} \lambda x\,.\,e & x = y \\ \lambda x\,.\,[v/y]e & \texttt{else} \end{cases}$$

$$[v/y](e_1 e_2) = ([v/y]e_1)([v/y]e_2)$$

We can handle the problem case directly in our definition. *Check the bound variable before we substitute in the body of a function*

Is there still a problem?

$$[v/y]x = \begin{cases} v & x = y \\ x & \texttt{else} \end{cases}$$

$$[v/y](\lambda x\,.\,e) = \begin{cases} \lambda x\,.\,e & x = y \\ \lambda x\,.\,[v/y]e & \texttt{else} \end{cases}$$

$$[v/y](e_1 e_2) = ([v/y]e_1)([v/y]e_2)$$

Problem Case II

α

$$[y/x](\lambda w\,.\,x) = \qquad [y/x](\lambda y\,.\,x) =$$

$$\lambda w\,.\,[y/x]x = \qquad \lambda y\,.\,[y/x]x =$$

$$\lambda w\,.\,x \neq \lambda y\,.\,y$$

α

We're not replacing a bound variable, but we *are* substituting an expression that has variables which *became* bound

The variable y is said to be **captured** in this (incorrect) substitution

Free and Bound Variables

$$FV(x) = \{x\} \tag{1}$$

$$FV(\lambda x\,.\,e) = FV(e) \setminus \{x\} \tag{2}$$

$$FV(e_1 e_2) = FV(e_1) \cup FV(e_2) \tag{3}$$

Free and Bound Variables

$$FV(x) = \{x\} \tag{1}$$

$$FV(\lambda x\,.\,e) = FV(e) \setminus \{x\} \tag{2}$$

$$FV(e_1 e_2) = FV(e_1) \cup FV(e_2) \tag{3}$$

Def. A variable x is **free** in e if it does not appear **bound** by a λ**.** Formally**:**

Free and Bound Variables

$$FV(x) = \{x\} \tag{1}$$

$$FV(\lambda x\,.\,e) = FV(e) \setminus \{x\} \tag{2}$$

$$FV(e_1 e_2) = FV(e_1) \cup FV(e_2) \tag{3}$$

Def. A variable x is **free** in e if it does not appear **bound** by a λ. Formally:

1. x is free in x

Free and Bound Variables

$$FV(x) = \{x\} \tag{1}$$

$$FV(\lambda x\,.\,e) = FV(e) \setminus \{x\} \tag{2}$$

$$FV(e_1 e_2) = FV(e_1) \cup FV(e_2) \tag{3}$$

Def. A variable x is **free** in e if it does not appear **bound** by a λ. Formally:

1. x is free in x
2. x is free in $\lambda y.e$ if it is free in e and $x \neq y$

Free and Bound Variables

$$FV(x) = \{x\} \tag{1}$$

$$FV(\lambda x\,.\,e) = FV(e) \setminus \{x\} \tag{2}$$

$$FV(e_1 e_2) = FV(e_1) \cup FV(e_2) \tag{3}$$

Def. A variable x is **free** in e if it does not appear **bound** by a λ. Formally:

1. x is free in x
2. x is free in $\lambda y.e$ if it is free in e and $x \neq y$
3. x is free in $e_1 e_2$ if x is free in e_1 or e_2

Free and Bound Variables

$$FV(x) = \{x\} \quad (1)$$

$$FV(\lambda x\,.\,e) = FV(e) \setminus \{x\} \quad (2)$$

$$FV(e_1 e_2) = FV(e_1) \cup FV(e_2) \quad (3)$$

$FV(\lambda x . y\ x) =$

$FV(y\ x) \setminus \{x\}$

$(FV(y) \cup F(x)) \setminus \{x\}$

$\{y, x\} \setminus \{x\} = \{y\}$

Def. A variable x is **free** in e if it does not appear **bound** by a λ. Formally:

1. x is free in x
2. x is free in $\lambda y . e$ if it is free in e and $x \neq y$
3. x is free in $e_1 e_2$ if x is free in e_1 or e_2

Def. A variable x is **free** in e if $x \in FV(e)$ as above

Definition (Third Attempt)

$$[v/y]x = \begin{cases} v & x = y \\ x & \texttt{else} \end{cases}$$

$$[v/y](\lambda x\,.\,e) = \begin{cases} \lambda x\,.\,e & x = y \\ \lambda z\,.\,[v/y][z/x]e & x \in FV(v) \\ \lambda x\,.\,[v/y]e & \texttt{else} \end{cases}$$

$$[v/y](e_1 e_2) = ([v/y]e_1)([v/y]e_2)$$

Definition (Third Attempt)

$$[v/y]x = \begin{cases} v & x = y \\ x & \texttt{else} \end{cases}$$

$$[v/y](\lambda x\,.\,e) = \begin{cases} \lambda x\,.\,e & x = y \\ \lambda z\,.\,[v/y][z/x]e & x \in FV(v) \\ \lambda x\,.\,[v/y]e & \texttt{else} \end{cases}$$

$$[v/y](e_1 e_2) = ([v/y]e_1)([v/y]e_2)$$

Since we're interested in α-equivalence, we can first *replace* the bound variable and *substitute* it in the body of the function. This is called **α-renaming**

Definition (Third Attempt)

$$[v/y]x = \begin{cases} v & x = y \\ x & \texttt{else} \end{cases}$$

$$[v/y](\lambda x\,.\,e) = \begin{cases} \lambda x\,.\,e & x = y \\ \lambda z\,.\,[v/y][z/x]e & x \in FV(v) \\ \lambda x\,.\,[v/y]e & \texttt{else} \end{cases}$$

$$[v/y](e_1 e_2) = ([v/y]e_1)([v/y]e_2)$$

Since we're interested in α-equivalence, we can first *replace* the bound variable and *substitute* it in the body of the function. This is called **α-renaming**

Is there still a problem?

Problem Case III

$$FV(x) = \{x\}$$
$$FV(\lambda x\,.\,e) = FV(e) \setminus \{x\}$$
$$FV(e_1 e_2) = FV(e_1) \cup FV(e_2)$$

$$[v/y]x = \begin{cases} v & x = y \\ x & \texttt{else} \end{cases}$$
$$[v/y](\lambda x\,.\,e) = \begin{cases} \lambda x\,.\,e & x = y \\ \lambda z\,.\,[w/z][z/x]e & x \in FV(v) \\ \lambda x\,.\,[v/y]e & \texttt{else} \end{cases}$$
$$[v/y](e_1 e_2) = ([v/y]e_1)([v/y]e_2)$$

$$[x/y](\lambda x\,.\,xyz) =$$

$$[x/y](\lambda \boxed{z}.\,xyz) =$$

$$\lambda \boxed{z}.\,xxz$$

This isn't exactly a problem, but *we have to be careful about which variable to replace the bound variable* x *with*

If we choose z, then we capture a *different* variable!

"Correct" Definition

$$[v/y]x = \begin{cases} v & x = y \\ x & \texttt{else} \end{cases}$$

$$[v/y](\lambda x\,.\,e) = \begin{cases} \lambda x\,.\,e & x = y \\ \lambda z\,.\,[v/y][z/x]e & x \in FV(v), z \texttt{ is fresh} \\ \lambda x\,.\,[v/y]e & \texttt{else} \end{cases}$$

$$[v/y](e_1 e_2) = ([v/y]e_1)([v/y]e_2)$$

"Correct" Definition

$$[v/y]x = \begin{cases} v & x = y \\ x & \texttt{else} \end{cases}$$

$$[v/y](\lambda x\,.\,e) = \begin{cases} \lambda x\,.\,e & x = y \\ \lambda z\,.\,[v/y][z/x]e & x \in FV(v), z \texttt{ is fresh} \\ \lambda x\,.\,[v/y]e & \texttt{else} \end{cases}$$

$$[v/y](e_1 e_2) = ([v/y]e_1)([v/y]e_2)$$

Finally a definition, that works. Sort of...

"Correct" Definition

$$[v/y]x = \begin{cases} v & x = y \\ x & \texttt{else} \end{cases}$$

$$[v/y](\lambda x\,.\,e) = \begin{cases} \lambda x\,.\,e & x = y \\ \lambda z\,.\,[v/y][z/x]e & x \in FV(v), z \texttt{ is fresh} \\ \lambda x\,.\,[v/y]e & \texttt{else} \end{cases}$$

$$[v/y](e_1 e_2) = ([v/y]e_1)([v/y]e_2)$$

Finally a definition, that works. Sort of...

The only problem with this definition is that it now poses an <u>implementation issue</u>. **How do we come up with z?**

"Correct" Definition

$$[v/y]x = \begin{cases} v & x = y \\ x & \texttt{else} \end{cases}$$

$$[v/y](\lambda x\,.\,e) = \begin{cases} \lambda x\,.\,e & x = y \\ \lambda z\,.\,[v/y][z/x]e & x \in FV(v), z \texttt{ is fresh} \\ \lambda x\,.\,[v/y]e & \texttt{else} \end{cases}$$

$$[v/y](e_1 e_2) = ([v/y]e_1)([v/y]e_2)$$

Finally a definition, that works. Sort of...

The only problem with this definition is that it now poses an <u>implementation issue</u>. **How do we come up with z?**

In mathematics, we can say it's **always possible** to come up with a variable z, but when we're implementing a programming language, we need an *actual* procedure

Well-Scopedness and Closedness

open

$$\lambda x . y$$

closed

$$\lambda x . \lambda y . y$$

Def. *(informal)* An expression e is **well-scoped** if every free variable in e is "in scope" (more on that on Thursday)

Def. An expression e is **closed** if it has no free variables

Every closed term is well-scoped

One Solution: Well-Scopedness Check

$$[v/y]x = \begin{cases} v & x = y \\ x & \texttt{else} \end{cases}$$

$$[v/y](\lambda x\,.\,e) = \begin{cases} \lambda x\,.\,e & x = y \\ \lambda z\,.\,[v/y][z/x]e & x \in FV(v), z \texttt{ is fresh} \\ \lambda x\,.\,[v/y]e & \texttt{else} \end{cases}$$

$$[v/y](e_1 e_2) = ([v/y]e_1)([v/y]e_2)$$

One Solution: Well-Scopedness Check

$$[v/y]x = \begin{cases} v & x = y \\ x & \texttt{else} \end{cases}$$

$$[v/y](\lambda x\,.\,e) = \begin{cases} \lambda x\,.\,e & x = y \\ \lambda z\,.\,[v/y][z/x]e & x \in FV(v), z \texttt{ is fresh} \\ \lambda x\,.\,[v/y]e & \texttt{else} \end{cases}$$

$$[v/y](e_1 e_2) = ([v/y]e_1)([v/y]e_2)$$

If we only work with closed (well-scoped) expressions, then we don't need to worry about captured variables

One Solution: Well-Scopedness Check

$$[v/y]x = \begin{cases} v & x = y \\ x & \texttt{else} \end{cases}$$

$$[v/y](\lambda x\,.\,e) = \begin{cases} \lambda x\,.\,e & x = y \\ \lambda z\,.\,[v/y][z/x]e & x \in FV(v), z \texttt{ is fresh} \\ \lambda x\,.\,[v/y]e & \texttt{else} \end{cases}$$

$$[v/y](e_1 e_2) = ([v/y]e_1)([v/y]e_2)$$

If we only work with closed (well-scoped) expressions, then we don't need to worry about captured variables

The condition requiring α-renaming never holds!

demo

(lambda calculus)

Summary

The **lambda calculus** is a simple function programming language useful for learning basic functional concepts

Different **evaluation strategies** have different benefits (and may yield different programming behaviors)

Substitution is a bit tricky to define correctly (!) but any definition must preserve α–equivalence